

[illegible]

```

: constant TREE := D (XD_HEADER, DEF); SOURCE_NAME : TREE := D (XD_SOURCE_NAME, DEF); begin if HEADER.TY = DN_FUNCTION_SPEC then
return GET_BASE_TYPE (D (AS_NAME, ~HEADER)); elsif SOURCE_NAME.TY in CLASS_OBJECT_NAME then return GET_BASE_TYPE (D (SM_OBJ_TYPE, D (XD_SOURCE
_NAME, DEF))); elsif SOURCE_NAME.TY in CLASS_TYPE_SPEC then if GET_BASE_TYPE (SOURCE_NAME).TY /= DN_TASK_SPEC then Put_Line ("!! NON
TASK TYPE NAME IN CALL TO GET_DEF_EXP_TYPE"); raise Program_Error; end if; return GET_BASE_TYPE (SOURCE_NAME); else ret
urn TREE_VOID; end if; end GET_DEF_EXP_TYPE;
-- function GET_BASE_TYPE (TYPE_SPEC_OR_EXP_OR_ID : TREE) return TREE is TYPE_SPEC : TREE := TYPE_SPEC_OR_EXP_OR_ID; begin
GET A TYPE_SPEC FOR THE EXPRESSION OR ID case TYPE_SPEC_OR_EXP_OR_ID.TY is when DN_VOID => null; when DN_USED_NAME_ID =>
TYPE_SPEC := D (SM_DEFN, TYPE_SPEC); if TYPE_SPEC /= TREE_VOID then TYPE_SPEC := D (SM_TYPE_SPEC, ~TYPE_SPEC); end if; when
0130 CLASS_OBJECT_NAME => TYPE_SPEC := D (SM_OBJ_TYPE, TYPE_SPEC); when DN_FUNCTION_ID => -- (FOR-SLICE WHOSE PREFIX IS FUNCTION WITH AL
L DEFAULT ARGS) TYPE_SPEC := GET_BASE_TYPE (D (AS_NAME, D (SM_SPEC, ~TYPE_SPEC))); when DN_PROCEDURE_ID => -- (FOR-IDENTIFIER AS EXP
RESSION BEFORE OVERLOAD RESOLUTION) TYPE_SPEC := TREE_VOID; when DN_GENERIC_ID => -- (FOR-EITHER OF-THE ABOVE-CASES) if D (
XD_HEADER, GET_DEF_FOR_ID (TYPE_SPEC)).TY = DN_FUNCTION_SPEC then TYPE_SPEC := GET_BASE_TYPE (D (AS_NAME, ~D (SM_SPEC, TYPE_SPEC))); else
E_SPEC := TREE_VOID; end if; when CLASS_TYPE_NAME | CLASS_RANGE => TYPE_SPEC := D (SM_TYPE_SPEC, TYPE_SPEC); when CLASS_US
ED_OBJECT | CLASS_EXP_EXP | DN_ATTRIBUTE | DN_FUNCTION_CALL | DN_INDEXED | DN_SLICE | DN_ALL => TYPE_SPEC := D (SM_EXP_TYPE, TYPE_SPEC);
when DN_SELECTED => TYPE_SPEC := GET_BASE_TYPE (D (AS_DESIGNATOR, TYPE_SPEC)); when CLASS_TYPE_SPEC => null; when DN_DISC
RETE_SUBTYPE => TYPE_SPEC := GET_BASE_TYPE (D (AS_NAME, D (AS_SUBTYPE_INDICATION, TYPE_SPEC))); when DN_SUBTYPE_INDICATION => TY
PE_SPEC := D (SM_TYPE_SPEC, D (AS_NAME, ~TYPE_SPEC)); when CLASS_UNSPECIFIED_TYPE => null; when others => Put_Line ("!! BAD
PARAMETER FOR GET_BASE_TYPE"); raise Program_Error; end case; -- GET UNCONSTRAINED FOR CONSTRAINED TYPE -- (IN CASE CONSTR
0140 AINED-PRIVATE WITH FULL TYPE VISIBLE) if TYPE_SPEC.TY in CLASS_CONSTRAINED then TYPE_SPEC := D (SM_BASE_TYPE, TYPE_SPEC); end if;
-- GET FULL TYPE SPEC FOR PRIVATE-OR INCOMPLETE if TYPE_SPEC.TY in CLASS_PRIVATE_SPEC then if D (SM_TYPE_SPEC, TYPE_SPEC) /= TREE_VOID-the
n TYPE_SPEC := D (SM_TYPE_SPEC, TYPE_SPEC); end if; elsif TYPE_SPEC.TY = DN_INCOMPLETE then if D (XD_FULL_TYPE_SPEC, TYPE_SPEC
) /= TREE_VOID then TYPE_SPEC := D (XD_FULL_TYPE_SPEC, TYPE_SPEC); end if; end if; -- LOOP TO GET BASE TYPE -- $$$ $ 0
K? NON-TASK --> PRIVATE ? while TYPE_SPEC.TY in CLASS_NON_TASK and then D (SM_BASE_TYPE, TYPE_SPEC) /= TYPE_SPEC loop TYPE_SPEC := D (SM_BAS
E_TYPE, TYPE_SPEC); end loop; return TYPE_SPEC; end GET_BASE_TYPE;
-- function GET_BASE_PACKAGE (PACKAGE_ID : TREE) return TREE is UNIT_DESC : TREE := D (SM_UNIT_DESC, PACKAGE
_ID); BASE_ID : TREE; begin if UNIT_DESC.TY = DN_RENAMES_UNIT then BASE_ID := GET_NAME_DEFN (D (AS_NAME, UNIT_DESC)); if BASE_
_ID /= TREE_VOID then return GET_BASE_PACKAGE (BASE_ID); end if; end if; return PACKAGE_ID; end GET_BASE_PACKAGE;
end DEF_UTIL;

```